

Simulación en MatLab del movimiento y la detección de obstáculos de un robot.

Alejandro Otero Vázquez Robótica y Mecatrónica. Máster NTEyF UCM

Este trabajo busca replicar el comportamiento de un robot diferencial. La simulación tratará de replicar un algoritmo estadístico de construcción de un mapa mediante la detección de obstáculos imprevistos. El programa se divide en dos objetivos: emular el funcionamiento de un encoder para conocer la posición del robot y el funcionamiento de un sensor de distancia que detecte los objetos.

1 Movimiento del robot en un plano.

El programa *movRobot.m* implementa dentro de sí toda la arquitectura del robot. Se definen tres estructuras *Map*, *rob* y *sensor* relativas al espacio por el que se moverá el robot y a los parámetros que definen el robot respectivamente. Dentro de la estructura *Map* de almacenarán dos mapas: *Map.M*, que contiene el mapa "real" por el que se moverá el robot y *Map.sen* que será el mapa que se construirá a partir de las medidas de los sensores. *Map.M* es una matriz de 0s y 1s donde '0' significa celda con espacio libre y '1' significa celda ocupada por un obstáculo. Típicamente se trabajará con mapas importados de archivos externos almacenados en un fichero '*Mapa.dat*'. Para esta simulación se ha utilizado el mapa de la figura1. El mapa que se construirá

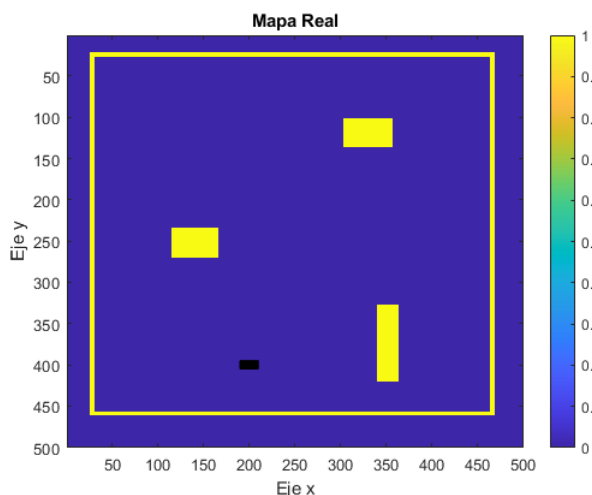


Figure 1: Mapa empleado para la simulación.

durante la simulación *Map.sen* se inicializa con todas las celdas a 0.5. Se establece 0.5 como un valor de incertidumbre en el que todavía no se conoce a ciencia cierta si habrá o no obstáculo.

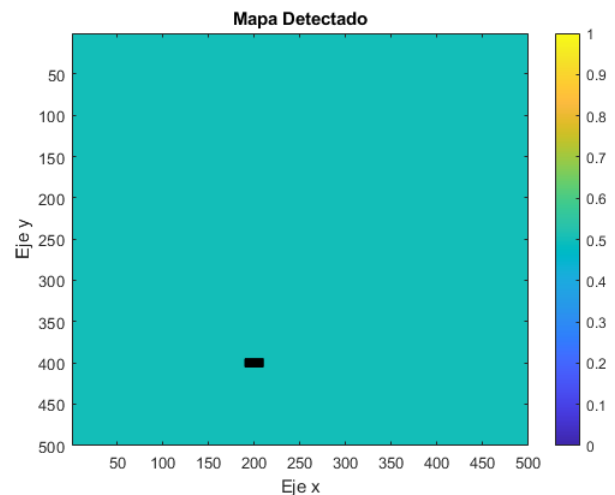


Figure 2: Condiciones iniciales para el mapa medido por los sensores.

1.1 Consideraciones Iniciales.

Los datos relativos a las dimensiones del robot y parámetros del sensor de distancia se almacenan en sendas estructuras que almacenan la siguiente información:

————— Model Robot —————

`rob.Nc=100; % Encoder: total pulses`

`rob.Rb=5; % wheel radius (cm)`

```

rob.b=10; % Distance between wheels (cm)
rob.x(1)=200;
rob.y(1)=400; % Position of robot
rob.theta(1)=0; % Orient of robot

```

```

————— Model Sensor data —————
sensor.rangTetha=30*pi/180;
% Beam: Width (rad)
sensor.rangR=80; % Beam: Range (cm)
sensor.epsil=10; % error (cm)
sensor.kd=3;

```

1.2 Lectura de los encoders.

La información que simula la medida de los encoders se almacena en un archivo. Este archivo típicamente se llamará '*Encoder.dat*'. El fichero contiene las cuentas tanto de la rueda izquierda como derecha almacenadas en dos columnas. Las ecuaciones (1)(2)(3) transforman las cuentas en una trayectoria (x, y, θ). Esta trayectoria simula la que debería seguir el robot para que el sensor encoder almacenara los datos que contiene.

```

for i=1:length(Enc.Nl)
    rob.x(i+1)=rob.x(i)+(Enc.Nl(i)+
    Enc.Nr(i))*(pi/20)*cos(rob.theta(i));
    rob.y(i+1)=rob.y(i)-(Enc.Nl(i)+
    Enc.Nr(i))*(pi/20)*sin(rob.theta(i));
    rob.theta(i+1)=rob.theta(i)+
    (Enc.Nr(i)-Enc.Nl(i))*pi/100;
end

```

La distancia recorrida por el robot es proporcional a la media de las cuentas medidas por los dos sensores. Este método puede llegar a cometer grandes errores de odometría al acumular cierto número de medidas. El punto final del robot depende en gran medida de si la desviación entre las cuentas de una rueda con otra se produjo al inicio del recorrido o al final. Si se muestrearan los sensores a mayor frecuencia, al recalculer la orientación del robot en periodos más cortos, estos errores podrían reducirse. Eliminarlos sería

imposible en cualquier caso. Existen dos opciones, la primera es tener en cuenta que la desviación de las cuentas se produce siempre al inicio de cada periodo, de modo que x e y se dependen de un θ calculado al inicio de cada periodo. Otra opción es lo contrario, suponer que el robot se mueve manteniendo la orientación con la que terminó en el muestreo anterior y recalculer θ al final. Como puede verse en las figuras la desviación es

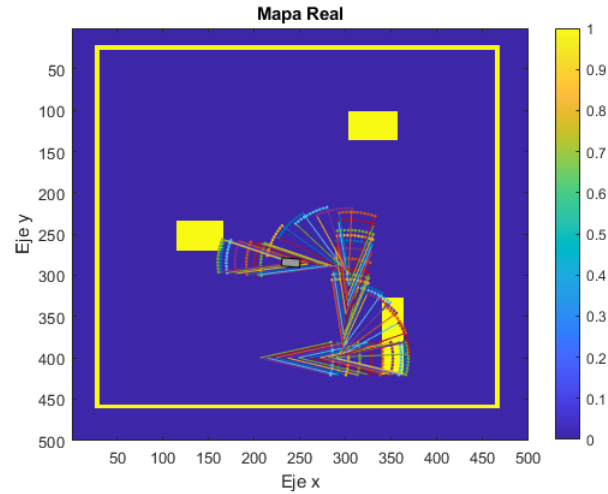


Figure 3: Trayectoria del robot calculando la orientación al inicio de cada muestra.

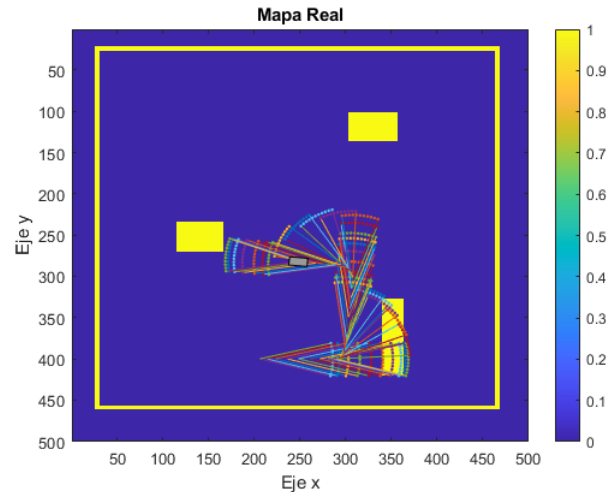


Figure 4: Trayectoria del robot calculando la orientación al final de cada muestra..

prácticamente despreciable dada una convención u otra. Ninguna se ajusta mejor a la realidad que la otra. Si el único dato del que se dispone es el número de cuentas es una incertidumbre que habrá que tener en cuenta en

las simulaciones.

Las siguientes figuras muestran una trayectoria más realista. Para calcularla se ha utilizado un fichero '*Encoder2.dat*' que tiene en cuenta los errores en las cuentas que pueda cometer el sensor. Estos errores se calcula que típicamente pueden llegar a ser del 7%.

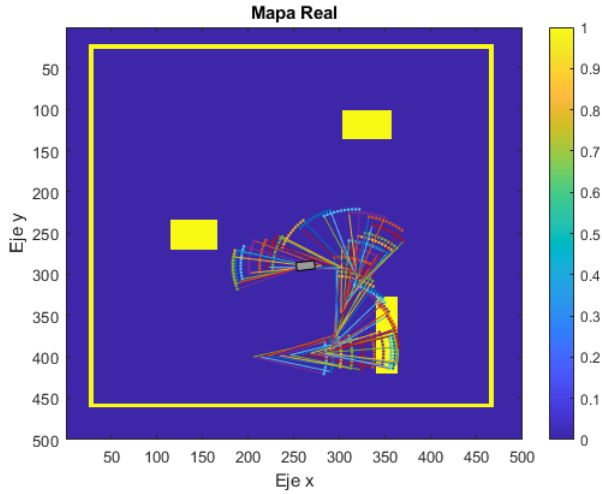


Figure 5: Trayectoria realista del robot calculando la orientación al inicio de cada muestra.

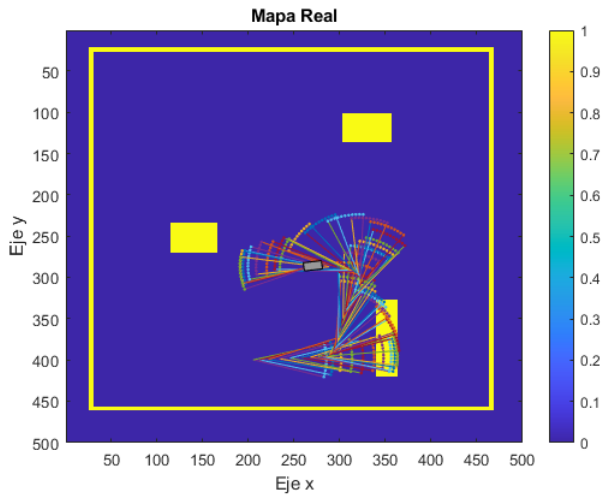


Figure 6: Trayectoria realista del robot calculando la orientación al final de cada muestra..

2 Medida de obstáculos imprevistos durante la trayectoria.

El programa *ultrasonidos.m* simula la medida de un sensor de distancia. Sin entrar en detalle sobre el funcionamiento interno del sensor, que podría funcionar por tiempo de vuelo, triangulación, etc. El modelo del sensor tiene en cuenta errores de odometría y errores intrínsecos a la medida. Es por ello que la información recibida por el robot se tratará de forma estadística. De esta forma las celdas tenderán a valores límite que dependerán de si a lo largo de las iteraciones el sensor ha detectado en ellas un objeto o no. Además, como estamos trabajando con un sensor de distancia, en cada iteración, las celdas tendrán simetría de revolución. Solo se puede calcular la distancia del objeto al sensor, no su orientación.

Teniendo esto en cuenta, el modelo define el valor de una celda entre $[0,1]$ en una iteración, ligado al valor de la celda en la iteración anterior como: Por teoría

$$P(m_{ij} | z) = \begin{cases} 0.5 + 0.5 / k_d & \text{si } r \in [r_0, r_0 + \varepsilon] \text{ y } \theta \in [\theta_1, \theta_2] \\ 0.1 & \text{si } r < r_0 \text{ y } \theta \in [\theta_1, \theta_2] \\ 0.5 & \text{Otro caso} \end{cases}$$

te probabilidad por lo tanto, la probabilidad acumulada para una celda, se calcula a partir de la probabilidad acumulada en la iteración anterior y la probabilidad condicionada de la ecuación anterior substituyendo $P(m_{ij}|z)$ como $P(Occ|z_t)$ en la siguiente ecuación: Este algoritmo puede implementarse en MatLab de la

$$P(Occ | z_{t_r}) = \frac{P(Occ | z_t) \cdot P(Occ | z_{t_{r-1}})}{P(Occ | z_t) \cdot P(Occ | z_{t_{r-1}}) + (1 - P(Occ | z_t)) \cdot (1 - P(Occ | z_{t_{r-1}}))}$$

siguiente forma:

```

MapOut=Map.sen;
MapaReal=Map.M;
Mr=length(valrange);    k=1;
[f,c]=size(Map.M);

r0=sensor.rangR+sensor.epsil;
for k=linspace(Mr,1,Mr)
    ptoArc=[posX+valrange(k)*cos(valtheta);posY-valrange(k)*sin(valtheta)]'
    for a=1:length(valtheta)
        if (MapaReal(floor(ptoArc(a,1))*c+floor(ptoArc(a,2)))==1)
            r0=sqrt((ptoArc(a,1)-posX)^2+(-ptoArc(a,2)+posY)^2)
        end
    end
end
end
for k=1:Mr

    ptoArc=[posX+valrange(k)*cos(valtheta);posY-valrange(k)*sin(valtheta)]';
    for i=1:length(valtheta)

        if (sqrt((ptoArc(i,1)-posX)^2+(ptoArc(i,2)-posY)^2)<r0)
            MapOut(floor(ptoArc(i,1))*c+floor(ptoArc(i,2)))=(MapOut(floor(ptoArc(i,1))
*c+floor(ptoArc(i,2)))*0.1) / ((MapOut(floor(ptoArc(i,1))
*c+floor(ptoArc(i,2)))*0.1)+(1-MapOut(floor(ptoArc(i,1))
*c+floor(ptoArc(i,2))))*(1-0.1));

        elseif(r0<=sqrt((ptoArc(i,1)-posX)^2+(ptoArc(i,2)-posY)^2))
            &&(sqrt((ptoArc(i,1)-posX)^2+(ptoArc(i,2)-posY)^2)<=r0+sensor.epsil)
            MapOut(floor(ptoArc(i,1))*c+floor(ptoArc(i,2)))=(MapOut(floor(ptoArc(i,1))
*c+floor(ptoArc(i,2)))*(0.5+0.5/sensor.kd)) / ((MapOut(floor(ptoArc(i,1))
*c+floor(ptoArc(i,2)))*(0.5+0.5/sensor.kd))+(1-MapOut(floor(ptoArc(i,1))
*c+floor(ptoArc(i,2))))*(1-(0.5+0.5/sensor.kd)));

        else
            MapOut(floor(ptoArc(i,1))*c+floor(ptoArc(i,2)))=(MapOut(floor(ptoArc(i,1))
*c+floor(ptoArc(i,2)))*0.5) / ((MapOut(floor(ptoArc(i,1))
*c+floor(ptoArc(i,2)))*0.5)+(1-MapOut(floor(ptoArc(i,1))*c+
floor(ptoArc(i,2))))*(1-0.5));
        end
    end
end
end

```

El algoritmo recorre dentro de un bucle for el vector $\text{valrange}(k)$ que contiene las posibles distancias desde 0 al rango del sensor a las que se pueden detectar objetos. El arco de celdas que están a una distancia $\text{valrange}(k)$ se almacena en una matriz ptoArc de dos columnas con las coordenadas x e y de las celdas.

En cada iteración, se evalúa si en alguna de las celdas de ptoArc hay un objeto que se declara dentro del mapa real. Para ello se hace un barrido por todas las celdas de la matriz ptoArc . En caso de que esto sea así, se establece que r_0 será la distancia desde el sensor al punto donde se ha detectado un objeto.

Habiendo calculado ya r_0 , se recorre de nuevo con un doble bucle for cada arco de celdas que equidistan al sensor a una distancia de $\text{valrange}(k)$ para cada distancia k . De este modo, dependiendo del valor de la distancia al sensor y de r_0 se asigna un nuevo valor estadístico a la probabilidad de que haya un objeto en cada celda siguiendo las ecuaciones de $P(m_{ij}|z)$.

Los resultados obtenidos midiendo con el sensor una vez por cada cuenta de los encoders son los siguientes (la nomenclatura establece que N es el número de medidas del sensor de distancia que se hacen para cada cuenta de los encoders):

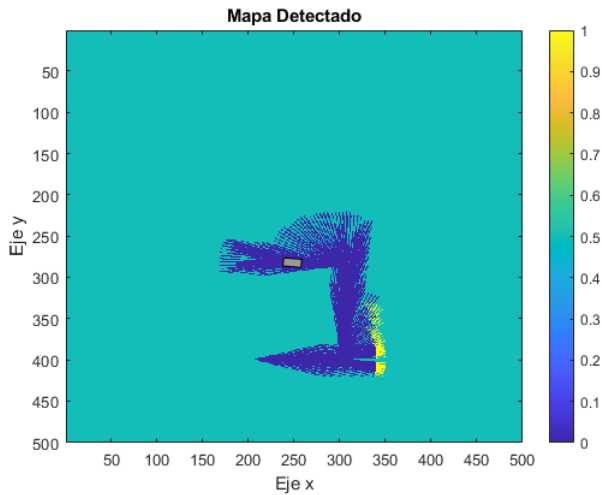


Figure 7: Mapa medido para valores teóricos de los encoders con $N=1$.

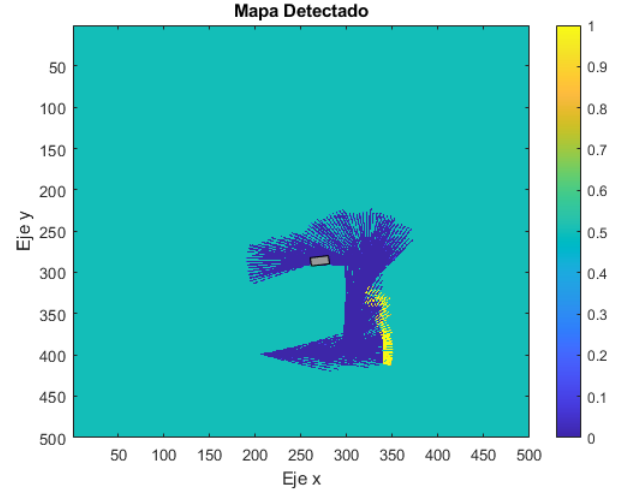


Figure 8: Mapa medido para valores experimentales de los encoders con $N=1$.

Se observa como con sucesivas iteraciones el mapa medido tiende prácticamente al mapa real teórico con el que se trabaja. Además, como ya se introdujo anteriormente, el método probabilístico elimina los errores de odometría que puedan tener los encoders. Aunque la trayectoria en el segundo caso no sea favorable, el mapa reconstruido por el sensor es igualmente fiable que el que no tiene en cuenta errores en las cuentas. Finalmente, se simula una medida del sensor de distancia que tenga un número mucho mayor de iteraciones. De esta forma, el mapa reconstruido va a tender a valores más precisos en sus celdas. Simulando el recorrido con $N=3$ se obtiene lo siguiente:

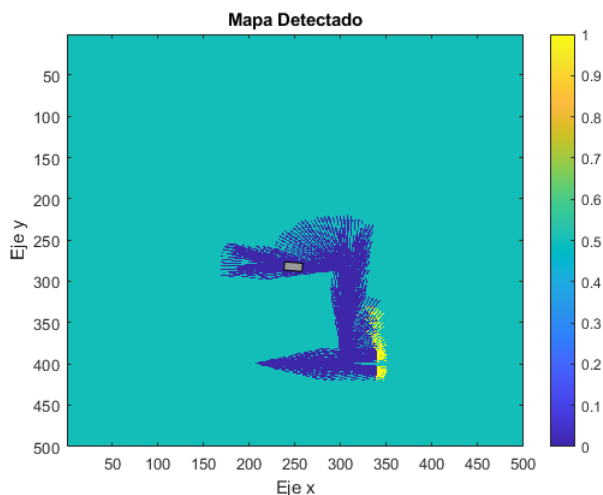


Figure 9: Mapa medido para valores teóricos de los encoders con $N=3$.

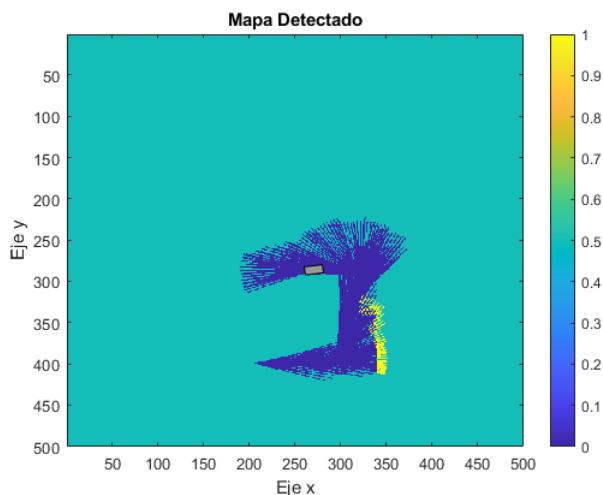


Figure 10: Mapa medido para valores experimentales de los encoders con $N=3$.

Las diferencias no son muy notables, lo que quiere decir que el sistema converge con rapidez a valores límite. Esto es muy favorable porque con pocas iteraciones el método estadístico es capaz de dar resultados muy sólidos.

3 Filtrado de señal ruidosa

En este segundo ejercicio se busca filtrar una señal ruidosa mediante un filtro FIR. Este filtro deberá ser capaz de eliminar el ruido de alta frecuencia de la señal.

3.1 Cálculo de la transformada de fourier de la señal.

Para analizar el espectro de frecuencias que tiene la señal se emplea la transformada de fourier. Esta transformada da información de que pesos tiene cada componente en frecuencias sobre la señal final. El objetivo del filtro FIR será atenuar las componentes a frecuencias altas que presente la señal. La siguiente figura muestra el valor absoluto de la transformada rápida de fourier de la señal, normalizada a la frecuencia de muestreo. Esta función es simétrica para frecuencias "negativas" por lo que solo se muestra el semieje positivo de frecuencias.

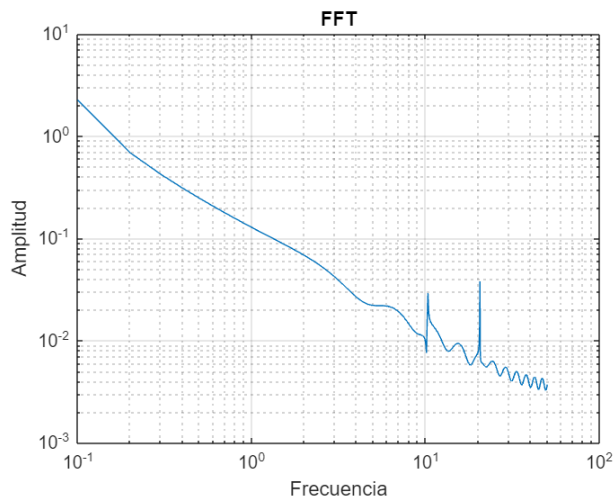


Figure 11: Transformada de fourier de la señal original.

Se observa como la señal tiene ciertas componentes entorno a la frecuencia de muestreo (50Hz).

3.2 Filtro FIR.

Los filtros FIR pasa baja se construyen a partir de una ventana que pondera los valores de la señal en cada punto con los valores de la señal en tiempos próximos. Los filtros FIR se caracterizan porque el coeficiente del

denominador cuando se construye el filtro es 1. El denominador caracteriza los polos de la señal. Si la señal tiene polos quiere decir que el filtro tiene en cuenta valores futuros de la señal para obtener la señal filtrada. Al presentar denominador 1, estos filtros aplican solo sobre los valores para tiempos pasados de la señal.

En este caso se ha decidido utilizar un filtro hanning de orden 50 para filtrar la señal. Este filtro tiene la siguiente forma. Como puede observarse, para frecuencias altas el filtro adopta valores muy cercanos a 0. Atenuando todas estas componentes en la señal resultante.

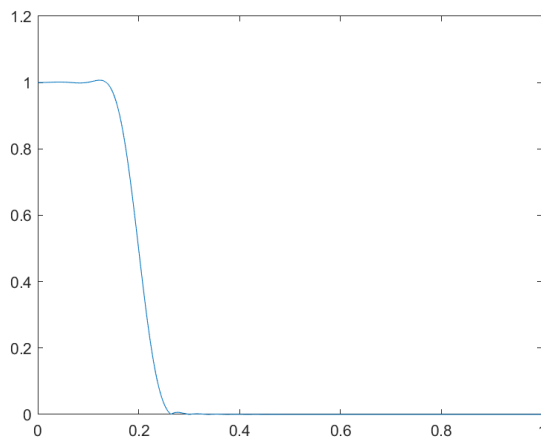


Figure 12: Ventana de filtro FIR Hanning.

3.3 Señal filtrada por FIR.

La siguiente figura muestra la comparativa entre la señal original y la señal después de aplicar un filtro pasa baja tipo FIR. Se observa como el ruido de alta frecuencia se ha visto atenuado sin comprometer los valores promedio de la señal. Una ventana mal construida podría haber atenuado las frecuencias bajas haciendo que la forma de la señal cambiase. En este caso, aunque todavía persiste cierta componente de alta frecuencia, se ha visto en gran medida atenuada.

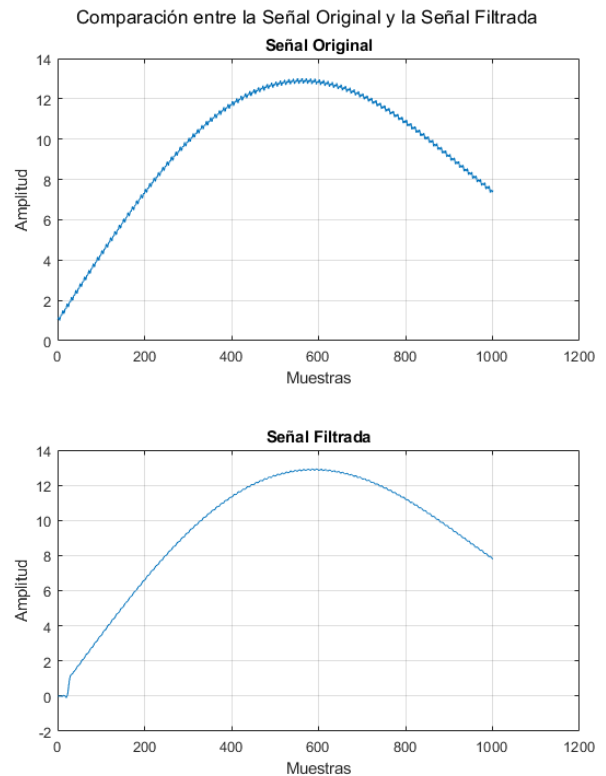


Figure 13: Comparación entre la señal filtrada con señal original tras aplicar un filtro FIR.

3.4 Señal filtrada por media móvil.

En robótica móvil es realmente común trabajar con sistemas empotrados que no son capaces de aplicar filtros de esta manera. Principalmente porque este tipo de aplicaciones deben actuar sobre la señal in-situ para tomar decisiones. Es por esto que es muy interesante implementar también un algoritmo de media móvil que filtre la señal de ruido. Este filtro funciona de forma muy simple. Básicamente cada vez que recibe una nueva medida de la señal pondera su valor haciendo una media con las 4 medidas anteriores. De esta forma se consigue un efecto de suavización de la señal que también brinda muy buenos resultados.

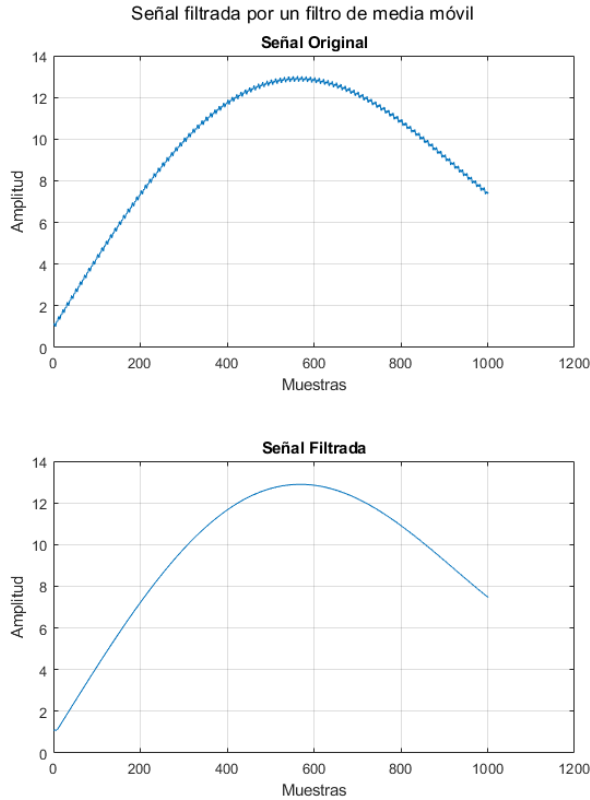


Figure 14: Comparación entre la señal filtrada con señal original aplicando un filtro de media móvil.

3.5 Comparación de filtros.

Es interesante comparar el espectro de frecuencias que presenta la señal cuando ha sido filtrada por distintos filtros. La siguiente figura muestra la transformada de fourier de la señal después de que se haya filtrado mediante un filtro FIR y mediante una simple media móvil.

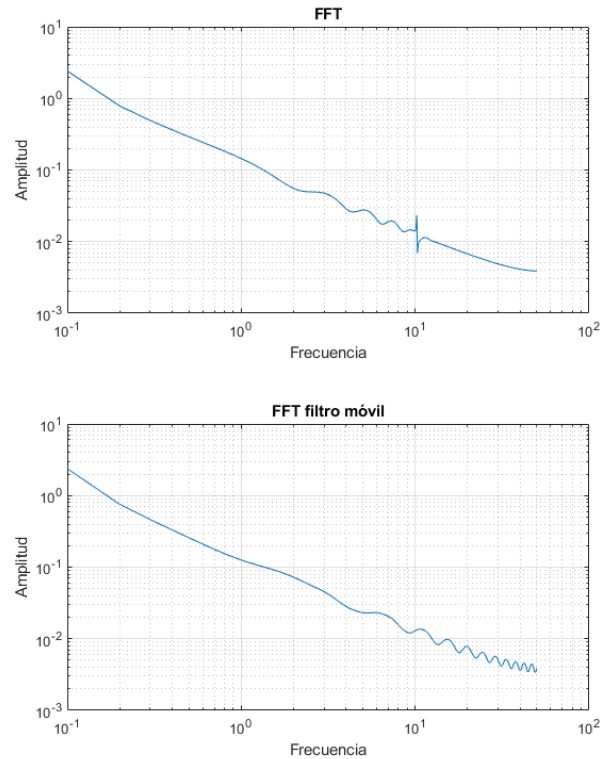


Figure 15: Comparación entre el espectro de frecuencias de la señal filtrada por dos filtros de origen distinto.

3.6 Anexo con el código empleado para los filtros.

Cargamos la señal en dos variables.

```
senal = load('senal.dat');  
t=senal(:,1);  
y=senal(:,2);
```

Se calcula la transformada de fourier de la señal. Para observar mejor la transformada de plotea sobre unos ejes logaritmicos.

```
n=length(t);  
fs = n/(t(end));  
fft_mel=abs(fft(y,2^nextpow2(n)));  
frec=linspace(0,fs/2,n/2+1);  
fft_mel=fft_mel(1:length(frec))/n;%Nos quedamos solo con los datos de  
frecuencia positivos  
figure()  
loglog(frec,fft_mel);  
ylabel('Amplitud')  
xlabel('Frecuencia')  
title('FFT');  
grid;  
hold off
```

Se crea la ventana del filtro y se aplica sobre la señal ruidosa.

```
fc = 10/(fs/2);  
N=50;  
hw = hanning(N+1);  
h = fir1(N,fc,hw);  
[hw,wd]=freqz(h,1); plot(wd/pi,abs(hw));
```

```
yf = filter(h,1,y);  
figure;  
subplot(2,1,1);  
plot(y);  
title('Señal Original');  
xlabel('Muestras');  
ylabel('Amplitud');  
grid on;
```

```
subplot(2,1,2);  
plot(yf);
```

```

title('Señal Filtrada');
xlabel('Muestras');
ylabel('Amplitud');
grid on;

% Mostrar las dos subtramas en la misma ventana
sgtitle('Comparación entre la Señal Original y la Señal Filtrada');

Implementación de un filtro móvil para emular el algoritmo que se
podría implementar en un sistema empotrado.
% Parámetros del filtro de media móvil
N = 50; % Longitud de la ventana de promedio móvil

% Implementación del filtro de media móvil
yf_real = zeros(size(n)); % Vector para almacenar la señal filtrada
for i = 1:length(y)
    if i <= N
        yf_real(i) = mean(y(1:i)); % Calcula el promedio de los primeros i elementos
    else
        yf_real(i) = mean(y(i-N+1:i)); % Calcula el promedio de los últimos N elementos
    end
end

% Graficar la señal de entrada y la señal filtrada
figure;
subplot(2,1,1);
plot(y);
title('Señal Original');
xlabel('Muestras');
ylabel('Amplitud');
grid on;

subplot(2,1,2);
plot(yf_real);
title('Señal Filtrada');
xlabel('Muestras');
ylabel('Amplitud');
grid on;

% Mostrar las dos subtramas en la misma ventana

```

```
sgtitle('Señal filtrada por un filtro de media móvil');
```

Se compara el espectro de frecuencias de la señal comparando los resultados de los dos métodos.

```
n=length(t);
fs = n/(t(end));
fft_mel=abs(fft(yf,2^nextpow2(n)));
fft_real=abs(fft(yf_real,2^nextpow2(n)));
frec=linspace(0,fs/2,n/2+1);
fft_mel=fft_mel(1:length(frec))/n;%Nos quedamos solo con los datos de
frecuencia positivos
fft_real=fft_real(1:length(frec))/n;
figure()
subplot(2,1,1);
loglog(frec,fft_mel);
ylabel('Amplitud')
xlabel('Frecuencia')
title('FFT');
grid;
subplot(2,1,2);
loglog(frec,fft_real);
ylabel('Amplitud')
xlabel('Frecuencia')
title('FFT filtro móvil');
grid on;
hold off
```